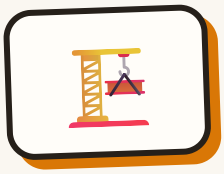


TERRAFORM STUDY SERIES · VOLUME ONE

Terraform Level 1 + 2

The complete **beginner-to-intermediate** guide: Infrastructure as Code, the core workflow, AWS provider setup, networking, compute, remote state — and the Day 2 patterns that turn a script into a team tool. Built on the course deck by **Eng. Mohamed Magdy**.

Part I · Day 1
Foundations · Workflow · State

Part II · Day 2
Variables · Workspaces · HCL

Part III · L2
Provisioners · Modules · Capstone

CONTENTS

🗺 Map of the Book

☑ tick as you go

Every page is a clickable door. Use the **TOC** badge at the bottom of every page to return here. Read the **Executive Summary** first — it tells the whole story in one page.

▣ How to read this book

- 🟢 **Concept pages** — read in order on Day 1.
- 🟡 **Mechanics** — when you want to understand what plan/apply/state actually do.
- 🔴 **Labs** — you need a terminal and 30 minutes.
- 🖨 **Print the cheat page** — review card before any interview.

EXECUTIVE SUMMARY

🗺️ The Route — One Page, Whole Book

📖 read first

We begin with **why** clicking through consoles doesn't scale, and meet the contractor who fixes it. From there you learn to draw blueprints: **AWS provider** brings the keys (without exposing them); **your first VPC** draws the boundary. The book's beating heart: **state**, the ledger that remembers every resource, guarded by the **plan**, the dress rehearsal.

Day 1 builds the foundation: **init** wires plugins, **plan/apply** executes the diff, **remote backend** stores the ledger, **locking** prevents collisions. By the end of Day 1 you've built a VPC, subnets, gateways, and a working EC2 with a security group — the whole cloud story in one sitting.

Day 2 turns the one-off script into a team tool: **variables** parameterize inputs, **workspaces** isolate state per env, **count & for_each** build N copies, **outputs** export values for CI/CD and other modules. One config, many environments, zero duplication.

Level 2 closes the loop: **providers** as translators to every cloud API, **provisioners** as the last-resort escape hatch (with doctrine), **modules** as Lego bricks that package reuse. The **CAPSTONE** stitches every chapter together in one hands-on build: provider → VPC → 2 subnets → 4 instances → security group → output IPs → provisioner → destroy.

> 🔑 **THE GOLDEN PATH IN 3 SENTENCES**
>
> We start with the **pain** (ClickOps doesn't scale), learn the **blueprint** (HCL + state + plan), and end with the **proof** (a working cloud environment that comes up identically every time).

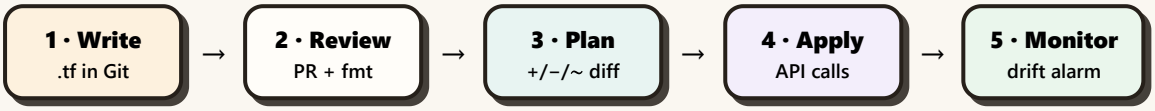
What is Infrastructure as Code?

6 MIN · T1

Manual server setup is the chef who "just knows" the dish — until they quit, and nobody can remake it. **Infrastructure as Code (IaC)** replaces tribal knowledge with a written recipe anyone can follow, with the same taste every time. `main.tf` is the blueprint; `terraform apply` is the construction crew.

> 🔑 **THE RULE**
> **Idempotence.** Apply the same code ten times → same infrastructure every time. `f(f(x)) = f(x)`. The property that makes IaC safe to retry, safe in CI, safe to share across teams.

The 5-step loop (memorize)



Code says where to go · state remembers where you've been · refresh checks where you are.

Why it exists — a real story

Startup: 3 devs, 1 EC2 made by clicking. Works. Team grows to 20, need dev/staging/prod + 50 servers. Without IaC: nobody remembers the security-group rules, dev works / prod fails, the engineer who built it left, audit can't trace port openings, region outage = 3 weeks of clicking. With IaC: same company rebuilds prod in 12 minutes via `terraform apply -var-file=prod.tfvars`, rolls back via `git revert`, proves every change with a PR.

❌ Manual (ClickOps)

Snowflake, tribal knowledge

Slow, error-prone

No rollback, no audit

Rebuild = weeks

✅ Infrastructure as Code

Every resource in code

Fast, parallel, automated

`git revert` + apply

Rebuild = minutes

> 😄 **THE RECIPE ANALOGY**
> ClickOps = chef who "just knows" the dish. IaC = written recipe anyone can follow. Bonus: anyone can read the recipe and improve it. Bonus bonus: you can compare two recipes (PRs) to see what changed.

⏭ **Next:** Now that you know WHY, meet the contractor — Chapter 2: What is Terraform + Why.

What is Terraform + The 6 Pillars of Why

7 MIN · T2 + T3

Terraform (HashiCorp, 2014, Go, MPL 2.0, open source) is a **multi-cloud standard** that builds, changes, and versions infrastructure safely. It manages existing providers (AWS, GCP, Azure, 3000+ more) and your own in-house solutions. Six reasons it wins, each with the objection already answered. 📝 2023+: new releases are BUSL — the MPL line continues as **OpenTofu**.

🔑 TERRAFORM IN ONE SENTENCE

Learn once, manage anything, with the best preview and biggest ecosystem. 3000+ providers. The community has answered every error you'll hit.

▣ The 6 pillars

1 · IaC done right

Code replaces clicks; Git replaces memory.

```
git log --oneline -- infra/
git revert <sha> && terraform apply
```

Objection "scripts do that" → scripts aren't idempotent/previewable; Terraform is.

2 · Platform-agnostic

```
resource "aws_s3_bucket" "a" { bucket = "x" }
resource "azurerm_resource_group" "b" { name = "rg" }
resource "github_repository" "c" { name = "infra" }
```

One `apply` touches 3 APIs. **Objection** "we're AWS-only" → still useful for GitHub/Cloudflare/Datadog in same run.

3 · Plan before apply

```
Plan: 1 to add, 1 to change, 0 to destroy.
  + aws_s3_bucket.new
  ~ aws_instance.web (t3.micro → t3.small)
```

PR shows the diff as a comment. `plan -out=tfplan` → `apply tfplan` = reviewed == executed.

4 · Coding best practices

```
terraform fmt -check && terraform validate && tflint
&& checkov -d .
```

Remote state+lock, no secrets, small modules, tag all — same bar as app code.

5 · Massive community

3000+ providers, 10k+ modules. Every error already answered. Hiring: most platform/DevOps posts name Terraform.

6 · Speed + reliable

Graph-parallel builds. Idempotent re-applies. Enterprise: remote runs, locking, policy gates.

😄 TF VS CFN — ONE-LINE

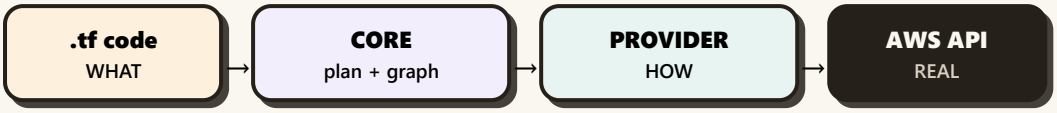
Unlocked phone + any carrier (more setup, works everywhere) vs **carrier-locked** (free, deep integration, only one network). Pick unlocked when you travel clouds.

⏭ **Next:** Look under the hood — Chapter 3: How Terraform Works (Core vs Plugins).

🧠 How Terraform Works — Core vs Plugins

7 MIN · T4 + T9

One-line: **Core reads your code and builds a graph; plugins do the dirty work talking to AWS/Azure/etc.** Core is the brain, providers are the hands. They communicate over gRPC (local RPC) — you almost never see the boundary, but knowing which side is the culprit saves hours.



State sits beside the core — its memory. `TF_LOG=DEBUG` reveals the gRPC chatter.

▣ What core does (brain)

Core job	Example
Parse HCL + variables + tfvars	`var.env` → `"prod"`
Build dependency graph	`subnet` → `vpc` (auto from references)
Order + parallelize	VPC first, then 3 subnets concurrently
Diff (plan)	code says v2, state says v1 → `~ update`
Manage state + lock	S3 + DynamoDB, one writer
Call plugins, never APIs	core never holds AWS keys — provider does

▣ Resource vs data source (the critical distinction)

Resource — managed

```
resource "aws_instance" "web" {
  ami = "ami-123"
  tags = { Name = "web" }
}
```

Terraform manages full lifecycle: create, update, delete. Counts in `plan`.

Data source — read-only

```
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]
  filter { name = "name" values =
["ubuntu/images/hvm-ssd/ubuntu-*-22.04*"] }
}
```

Lookup only — Terraform doesn't create or delete. Use for AMIs, AZs, secrets. Always fresh on every `plan`.

⚠ **THE "HIDDEN DEP" TRAP**

References handle 95% of ordering. For invisible deps (IAM eventual consistency, external side effects), use `depends_on = [...]` explicitly. `terraform graph | dot -Tpng > g.png` to visualize the order.

⏭ **Next:** The brain is built. Give it hands — Chapter 4: AWS Provider.

🔑 AWS Provider — Three Auth Patterns

7 MIN · T5

The `provider "aws"` block plus **credentials that never touch code** = Terraform talking to your account. The provider checks sources in order — first hit wins. Get auth right once, every guide after this just works.

> 🔑 **THE #1 RULE**
> **No keys in code. Ever.** State files + git history keep secrets forever. Static keys in the provider block is the #1 way that gets repos breached.

Auth priority order (first hit wins)

1 · provider block ❌ leaks → **2 · env vars** ✓ CI/local → **3 · ~/.aws + profile** ✓ humans → **4 · SSO / role / OIDC** ✓ teams

Three patterns you'll use

Pattern A · env vars (CI, scripts)

```
export AWS_ACCESS_KEY_ID="AKIA..."
export AWS_SECRET_ACCESS_KEY="..."
export AWS_REGION="us-east-1"
terraform plan
```

Pattern B · named profile (humans daily-driving)

```
$ aws configure --profile study
$ export AWS_PROFILE=study
$ terraform plan
```

Pattern C · OIDC in CI (no stored secrets)

```
permissions: { id-token: write }
- uses: aws-actions/configure-aws-credentials@v4
  with: { role-to-assume: arn:..., aws-region: us-east-1 }
```

Safety belt

```
provider "aws" {
  region = "us-east-1"
  allowed_account_ids = ["123456789012"]
  default_tags { tags = { Env = "study", ManagedBy = "terraform" } }
}
```

If creds accidentally point at prod, `terraform plan` fails fast. The 2-minute seatbelt that prevents the 2-day outage.

> ⚠️ **ERROR YOU'LL HIT**
> `No valid credential sources found` → nothing exported, no profile. Run `aws sts get-caller-identity` first; set `AWS_PROFILE` or env vars.

⏭ **Next:** Provider is wired. Draw your first real resource — Chapter 5: Your First Resource (aws_vpc).

Your First Resource — aws_vpc

8 MIN · T6

A VPC is the **plot of land** — a logically isolated virtual network where every other AWS resource lives. One per environment. Subnets divide it into AZs, route tables decide who talks to whom, gateways open the doors. Every EC2/RDS/ALB **must** launch in a VPC.

> 🔑 **VPC = NETWORK BOUNDARY**
> Pick a CIDR in private RFC1918 space. `10.0.0.0/16` = 65,536 IPs. AWS reserves 5 per subnet, so plan `/16` for VPC, `/24` per subnet. Default VPC exists — **never use it for new work.**

Resource block anatomy (memorize)

```
resource "aws_vpc" "main" {       //  ┌── type (RESOURCE_TYPE.LOCAL_NAME)
  cidr_block = "10.0.0.0/16"       //  │  argument: what you want
  enable_dns_hostnames = true      //  │
  enable_dns_support   = true      //  │
  instance_tenancy = "default"     //  │  "dedicated" = single-tenant $$
  tags = { Name = "study-vpc", Env = "dev" }
}
```

Every resource = `resource "TYPE" "LOCAL_NAME" { args }`. Local name is Terraform-only — AWS knows the resource by ID. Reference as `aws_vpc.main.id`, `aws_vpc.main.cidr_block`, etc.

The pro pattern — registry module beats hand-rolled

❌ Hand-rolled (200 lines)

You write every subnet, every route table, every association. Good for learning; bad for prod.

✅ Registry module (10 lines)

```
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "~> 5.1"   # pin!

  name = "study"
  cidr = "10.0.0.0/16"
  azs  = ["us-east-1a", "us-east-1b", "us-east-1c"]

  public_subnets   = ["10.0.1.0/24", "10.0.2.0/24",
"10.0.3.0/24"]
  private_subnets  = ["10.0.11.0/24", "10.0.12.0/24",
"10.0.13.0/24"]
  database_subnets = ["10.0.21.0/24", "10.0.22.0/24",
"10.0.23.0/24"]

  enable_nat_gateway = true
  single_nat_gateway = true   # dev only

  tags = { Env = "dev" }
}
```

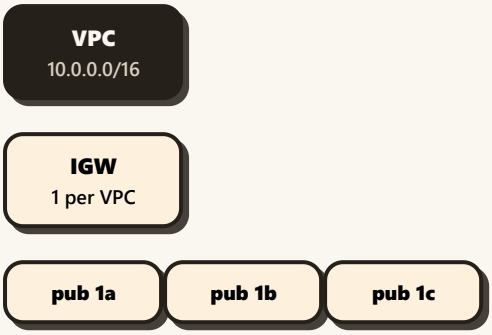
Rule: use the module in any real project. Hand-roll only when learning or you need a non-standard layout.

⏭ **Next:** The plot is drawn. Divide it into AZs and open the gates — Chapter 6: VPC Networking.

🌐 Subnets · Route Tables · IGW · NAT

8 MIN · T6 (deep)

VPC is the plot. **Subnets** divide it into AZs. **Route tables** decide who can talk to whom. **Internet Gateway** opens the public door. **NAT Gateway** lets private subnets sneak out for updates without being reachable from the internet.



Public subnets share a route table pointing 0.0.0.0/0 → IGW. Private subnets share one pointing 0.0.0.0/0 → NAT (in a public subnet). DB subnets have **no default route** — never.

▣ The 4 resources you touch every day

```
resource "aws_subnet" "public" {
  count                   = length(local.azs)
  vpc_id                  = aws_vpc.main.id
  cidr_block              = cidrsubnet(aws_vpc.main.cidr_block, 8, count.index)
  availability_zone       = local.azs[count.index]
  map_public_ip_on_launch = true
  tags = { Name = "public-${local.azs[count.index]}", Tier = "public" }
}
```

```
resource "aws_internet_gateway" "igw" { vpc_id = aws_vpc.main.id }
resource "aws_eip" "nat" { domain = "vpc" }
resource "aws_nat_gateway" "nat" {
  allocation_id = aws_eip.nat.id
  subnet_id     = aws_subnet.public[0].id
  depends_on    = [aws_internet_gateway.igw]
}
```

⚠️ **GOTCHA**

AWS reserves **5 IPs per subnet**. A `/24` = 251 usable. **Never hardcode AZs** — use `data "aws_availability_zones" + slice(..., 0, 3)`. A subnet is "public" only when its route table points 0.0.0.0/0 → IGW.

Day-2 ops

- ✅ Flow logs on the VPC
- ✅ 1 NAT/AZ in prod, 1 in dev
- ✅ Tag every subnet: Tier + AZ
- ✅ DB route table: **no default route, ever**
- ✅ Gateway endpoints for S3 + DynamoDB

Errors you'll hit

`CIDR overlaps with existing` → use `cidrsubnet`

`host bits set in CIDR` → `10.0.1.0/24` not `10.0.1.5/24`

`NAT in private subnet` → put in public

`cannot delete subnet: dependencies` → destroy in reverse

⏭ **Next:** Network is built. Now wire the engine — Chapter 7: terraform init.

terraform init — What It Really Does

6 MIN · T7

`terraform init` is the first command of any project — and the one people forget has **two jobs**, not one. It installs provider binaries AND wires up the backend (if you have one). No cloud calls; think `npm install` for infra.

> 💡 **SAFE TO RE-RUN**
> Init never touches real infra. Re-run it anytime you change `required_providers`, swap the backend block, or upgrade a provider. `-upgrade` bumps to newest allowed by `~>`. `-reconfigure` forgets the saved backend choice.

Job 1 (always) — install providers + modules

```
$ terraform init
Initializing the backend...
Initializing provider plugins...
- Installing hashicorp/aws v5.0.1
- Installed hashicorp/aws v5.0.1 (signed by HashiCorp)
- Installing hashicorp/local v2.5.1
- Installed hashicorp/local v2.5.1 (signed by HashiCorp)

Terraform has been successfully initialized!
```

What lands on disk: `.terraform/providers/.../terraform-provider-aws_v5.0.1` (the binary) + `.terraform.lock.hcl` (exact version + hashes, commit this).

Job 2 (only with a backend block) — wire up state storage

```
$ terraform init -migrate-state
Initializing the backend...
Backend type: s3
Do you want to copy existing state to the new backend? yes
Successfully configured the backend "s3"!

Terraform has been successfully initialized!
```

Init performs `HeadBucket` and `DescribeTable` during setup. **Missing bucket?** `NoSuchBucket` error. **Backend resources must exist BEFORE first init** — bootstrap folder with local state creates them.

The .gitignore canon

```
.terraform/           # downloaded provider binaries — always ignore
*.tfstate             # state file — has secrets, NEVER commit
*.tfstate.backup      # backup created by apply
*.tfvars              # real values — ignore
!example.tfvars       # except the example template
tfplan                # plan output binary
.terraform.lock.hcl   # COMMIT THIS — pins versions + hashes
```

> ⚠️ **THE LOCK FILE IS IN GIT**
> Counter-intuitive but critical: the lock file makes your team byte-identical. `terraform providers lock` regenerates hashes for all platforms. CI and laptop get the same provider binary.

⏭ **Next:** The engine is wired. The 7-command loop — Chapter 8: plan · apply · tfstate.

🔁 plan · apply · tfstate — The Core Loop

🟡 7 MIN · T8 + T10 + T15

Every Terraform session runs the same 7-command loop. `init` is the install (no cloud calls), `plan` is the dress rehearsal, `apply` is the real show, and a second `apply` proves idempotence by saying "No changes." State is the ledger written by `apply` and read by every subsequent `plan`.

> 🔑 **THE CONTRACT**
> **plan** is a contract, not a forecast. `plan -out=tfplan` + `apply tfplan` guarantees: what you reviewed = what was executed.
> **Never edit state by hand** — use `terraform state mv/rm` instead.

▣ The 7-command loop (every session)

```
terraform version    # 0. what toolchain?
terraform init       # 1. install providers + modules → .terraform/ + .lock.hcl
terraform fmt        # 2. canonical formatting (CI gate: fmt -check)
terraform validate   # 3. syntax check (offline, no cloud)
terraform plan       # 4. preview diff (+/~/-) — read this carefully
terraform apply      # 5. execute + write state
terraform apply      # 6. prove idempotence → "No changes"
```

Apply twice in a row: second one says No changes. That's idempotence made visible.

▣ What tfstate actually contains

```
{
  "version": 4,
  "terraform_version": "1.10.0",
  "serial": 12,
  "outputs": { "vpc_id": { "value": "vpc-0ab123", "type": "string" } },
  "resources": [
    {
      "mode": "managed",
      "type": "aws_vpc",
      "name": "main",
      "provider": "provider["registry.terraform.io/hashicorp/aws"]",
      "instances": [{
        "schema_version": 1,
        "attributes": {
          "id": "vpc-0ab123",
          "cidr_block": "10.0.0.0/16",
          "enable_dns_hostnames": true,
          "tags": { "Env": "dev", "Name": "study-vpc" }
        }
      }]
    }
  ]
}
```

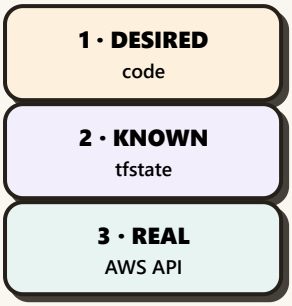
State stores: resource IDs, all attributes, dependencies, provider metadata. The mapping from your code addresses (e.g. `aws_vpc.main`) to real cloud IDs (e.g. `vpc-0ab123`).

⏭ **Next:** You have the loop. Now meet the three things it compares — Chapter 9: The 3-State Model.

The 3-State Model — code · state · reality

7 MIN · T11

Every plan is a 3-way comparison. Learn these three names — interviews ask them directly. **Desired state** is your `.tf` files. **Known state** is `terraform.tfstate`. **Real state** is what AWS looks like right now (after refresh).



Refresh = read REAL via API, compare to KNOWN. Diff = compare DESIRED to KNOWN. Anything off → plan shows +/~/−. State does 4 jobs: **track** (what's managed), **compare** (desired vs known), **metadata** (IDs/IPs/ARNs), **perform** (cache so plan stays fast).

Reading a plan output

```
Terraform will perform the following actions:

  + aws_instance.web              # create
  ~ aws_s3_bucket.notes           # update in-place
  -/+ aws_instance.replace         # destroy then create (force-new)
  - aws_security_group.old        # destroy

Plan: 1 to add, 1 to change, 1 to destroy.
```

Symbol	Means	Danger
+ create	in code, not in state	**safe**
~ update in-place	same ID, new attrs	**safe** (low)
-/+ destroy+create	attribute forces new	⚠️ data loss
- destroy	in state, gone from code	🔴 read every line

⚠️ **RENAME TRAP**

Terraform's identity is the **address** (`aws_instance.web`), not attributes. Rename the local name → destroy + create. **Use a `moved` block** for safe renames (preserves state, no rebuild).

⏭ **Next:** You have the model. Now store the ledger safely — Chapter 10: State Risks & Remote Backend.

8 MIN · T12

State Risks & Remote Backend (S3 + DynamoDB)

Local state dies with the laptop. The whole team needs **one** state, in a place that locks, versions, and survives. The cure: **S3 stores the JSON snapshots**; **DynamoDB holds the mutex**. Both, configured before the second human exists.

> 🔑 **THE 3 PAINS OF LOCAL STATE**
> ① Teammate applies from their laptop → your state is stale → resources duplicated. ② Two `apply` at once → corrupt JSON or half-written resources. ③ Laptop dies / file deleted → management history gone. **All three cured by S3 + DynamoDB.**

Backend anatomy

```
terraform {
  backend "s3" {
    bucket         = "my-study-tfstate-12345"   # unique
    key            = "dev/terraform.tfstate"    # per-env path
    region         = "us-east-1"
    dynamodb_table = "tfstate-locks"
    encrypt        = true
  }
}
```

S3 = JSON snapshots (versioned, encrypted). DynamoDB = mutex (conditional-write row). Two different stores, two different jobs.

Bootstrap (chicken-and-egg)

```
$ aws s3api create-bucket --bucket my-study-tfstate-12345
$ aws s3api put-bucket-versioning --bucket my-study-tfstate-12345     --versioning-configuration Status=Enabled
$ aws s3api put-bucket-encryption --bucket my-study-tfstate-12345     --server-side-encryption-configuration '{"Rules":
[{"ApplyServerSideEncryptionByDefault":{"SSEAlgorithm":"AES256"}}]}'
$ aws dynamodb create-table --table-name tfstate-locks     --attribute-definitions AttributeName=LockID,AttributeType=S     -
-key-schema AttributeName=LockID,KeyType=HASH --billing-mode PAY_PER_REQUEST
```

Pro pattern: keep a tiny `bootstrap/` folder whose ONLY job is creating bucket + table with local state, then never touch it again.

How locking works (3 steps)

1 · apply starts conditional-write LockID → **2 · second apply** Error: lock held → **3 · first apply ends** DELETE LockID

With lock (safe)

```
Shell A: terraform apply ..... (holds lock,
writing)
Shell B: terraform apply → Error: Error acquiring the
state lock
  Lock Info: ID xxx, Who: teammate@laptop, Created:
10:02:11
B waits or exits → no corruption.
```

Without lock (disaster)

```
10:00:00  state v1: { web }
10:00:20  YOU apply → v2: { web, db }
10:00:25  SARA apply → OVERWRITES → { web, cache }
10:00:26  db exists in AWS. Terraform forgot it.
→ ghost resource. bill grows quietly.
```

⏭ **Next:** You have the backend. Now the lock lifecycle — Chapter 11: State Locking.

State Locking — The Mutex Explained

7 MIN · T13

Two engineers + concurrent apply = corrupted state. The fix: a **DynamoDB lock table** holds a single row per backend key. First apply grabs it (conditional-write), others fail fast, lock releases when apply finishes. The contract: **only one writer to state at a time, ever.**

🔑 THE LOCK LIFECYCLE

apply starts → conditional-write LockID → if row exists, **fail fast** with who/when/operation → if acquired, **do work + write new state to S3** → on success, **delete the lock row**. The lock is a row, not a file.

What the lock error looks like

```
$ terraform apply
| Error: Error acquiring the state lock
|
| Error acquiring the state lock: ConditionalCheckFailedException:
|   Conditional Write failed: LockID = my-bucket/dev/terraform.tfstate
|
| Lock Info:
|   ID:        abc-123-def-456
|   Path:      my-bucket/dev/terraform.tfstate
|   Operation: OperationTypeApply
|   Who:       teammate@laptop
|   Created:   2026-09-07 10:02:11 UTC
|   Version:   1.5.0
|
| Terraform acquires a state lock to protect the state from being
| written by multiple users at the same time. Please resolve the issue
| by removing the lock manually.
```

Force-unlock — the danger zone

```
# Step 1: read the error — Who, Operation, Created
# Step 2: CONFIRM the holder is dead
#   $ ps aux | grep terraform       # on their machine
#   ask in chat / check CI run status
# Step 3: only then, release it
$ terraform force-unlock abc-123-def-456
Do you really want to force-unlock?  # interactive confirm
# CI flag (no prompt, dangerous):
$ terraform force-unlock -force abc-123-def-456

# Step 4: PROVE HEALTH before continuing
$ terraform plan   # expect sane diff, NOT "create the world"
```

⚠️ NEVER HAND-DELETE THE LOCK ROW

Tempting: open DynamoDB console, delete the row, done in 10 seconds. **Don't.** The command exists because the console bypasses all the safety checks (wrong table? wrong key? live lock?). The error message already told you exactly which row to target.

⏭ **Next:** You have the lock. Now the state surgery tools — Chapter 12: State Subcommands.

🔧 State Subcommands + CLI Extras

7 MIN · T14 + T16

The state is a JSON file. The `terraform state` subcommands let you **operate on it without recreating resources**. Refactor names, drop orphans, adopt existing infra. Other CLIs (`fmt`, `graph`) keep the code clean and the dependencies visible.

🔑 THE 5 STATE SUBCOMMANDS

State subcommands change **state, not infrastructure**. `state rm` stops managing without deleting. `state mv` renames without recreating. `import` adopts existing infra into management.

State subcommands (5 essential)

Command	What it does	When to use
`state list`	List every resource address in state	"What's actually managed here?"
`state show <addr>`	Show the full attributes of one resource	"What's the actual ID of this VPC?"
`state pull`	Download the raw state JSON to stdout	Backup, debugging, hand inspection
`state mv <from> <to>`	Rename in state (no recreate)	Refactor `web` → `app` without rebuilding
`state rm <addr>`	Stop managing (does NOT delete from AWS)	Adopt a manual resource, drop an orphan
`terraform import <addr> <id>`	Adopt existing infra into state	"Terraform forgot this VPC, here's the ID"

`import` adopts but does NOT write config for you. Write the matching `resource` block → `import` → `plan` till clean, or the next `plan` will try to recreate it.

The other CLIs you'll use

`fmt` — canonical formatting

```
$ terraform fmt -check   # CI gate: exit 1 if not formatted
$ terraform fmt          # fix in place
```

Run before every commit. No-op if already formatted. Add to pre-commit hook.

`graph` — dependency map

```
$ terraform graph | dot -Tsvg > graph.svg
# installs graphviz, generates SVG of dep graph
```

Visualize the actual order Terraform will create things in. Great for debugging hidden deps.

😅 THE RENAME TRAP, AGAIN

Renaming `aws_instance.web` → `aws_instance.app` in your `.tf` is a **destroy + create**. **Always use `state mv` first**, then rename in code. Or use a `moved` block (stateful rename, no recreate).

⏭️ **Next:** Day 1 closes. Day 2 opens: Variables & the precedence ladder.

5 MIN · T15

🧹 fmt & destroy — Canon Format, Safe Teardown

`terraform fmt` is your formatting pre-commit hook. `terraform destroy` is your teardown command — same plan machinery as apply, opposite direction. Both are gateable, both are CI-runnable, both deserve respect.

🔑 **DESTROY IS JUST APPLY IN REVERSE**

Same plan/apply loop, but every resource is destroyed instead of created. `terraform destroy` prompts for confirmation; `-auto-approve` skips it (use in CI, never in interactive dev).

fmt before → after

Before

```
resource  "aws_vpc"   "main"{
cidr_block="10.0.0.0/16"
   tags   =   {Name="x",Env="dev"}
}
```

After `fmt`

```
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"

  tags = {
    Env  = "dev"
    Name = "x"
  }
}
```

Two-space indent, aligned =, sorted tag keys, blank line between blocks. `terraform fmt -check` fails CI if not formatted (exit 1). `terraform fmt` rewrites in place.

destroy — the teardown pattern

```
$ terraform plan -destroy -out=kill.plan
# preview what's about to die (always plan first)

$ terraform apply kill.plan
# applies EXACTLY the destroy plan

$ terraform destroy
# interactive: prompts "Do you really want to destroy?"

$ terraform destroy -auto-approve
# CI: skips prompt — gate behind human approval
```

⚠️ **PERMANENCE**

destroy is **not** reversible. State is updated, AWS resources are gone, bills stop. For anything holding data (RDS, S3 buckets), use `prevent_destroy = true` in the resource block to fail the destroy with a clear error.

destroy in the lab vs prod

Lab / dev env

- Always destroy at end of session
- Saves $$, prevents zombie resources
- Use `-auto-approve` in CI

Prod

- Almost never run naked `destroy`
- Use `prevent_destroy = true` on critical resources
- Use `-target` to scope down to specific resources

⏭ **Next:** Day 1 done. Day 2: Variables.

Variables & the Precedence Ladder

8 MIN · T17 + T18

Same code, many environments. **Variables** parameterize inputs. The **precedence ladder** determines which value wins when multiple are provided. Memorize it: the **lowest priority wins**.

🔑 THE PRECEDENCE LADDER

defaults < `terraform.tfvars` (auto-loaded) < `-var` flag (CLI) < `TF_VAR_*` env vars (CI/CD + secrets). Use `.tfvars` per env, override at runtime, never edit code.

Variables — the full anatomy

```
# variables.tf
variable "env" {
  type        = string
  default     = "dev"
  description = "which environment we're targeting"

  validation {
    condition     = contains(["dev", "staging", "prod"], var.env)
    error_message = "env must be dev/staging/prod."
  }
}

variable "instance_type" {
  type    = string
  default = "t3.micro"
}

# main.tf — reference as var.NAME
resource "aws_instance" "web" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = var.instance_type
  tags          = { Env = var.env }
}
```

The 4 ways to pass variables

```
# 1. defaults in variables.tf (lowest priority)
# 2. terraform.tfvars (auto-loaded if present)
env             = "dev"
instance_type   = "t3.micro"

# 3. -var flag (one-off, highest)
$ terraform plan -var="env=prod" -var="instance_type=t3.small"

# 4. TF_VAR_* env vars (CI/CD + secrets — never in code!)
$ export TF_VAR_env=staging
$ export TF_VAR_db_password=$(cat ~/.dbpass)
```

Secrets rule: never in `.tf` or `.tfvars` (both end up in PRs). Use `TF_VAR_*` env vars from your CI secret store (GitHub Actions secrets, Vault, etc.).

😅 THE "WHICH ONE WINS" GOTCHA

You have `default = "dev"` in variables.tf AND `-var="env=prod"` on the CLI. **Prod wins** (higher priority, overrides default). Run `terraform console` and type `var.env` to see what's actually in scope right now.

⏭ **Next:** One config, many environments — Workspaces & Advanced HCL.

8 MIN · T19 + T20

🗂️ Workspaces & Advanced HCL (count · for_each)

One config, isolated state per env (**workspaces**). One block, N copies (**count**). One block, N named copies (**for_each**). These three patterns are how you stop duplicating config.

> 🔑 **THE RIGHT TOOL FOR N COPIES**
> **for_each** wins almost every time. Removing one named key doesn't churn the rest (stable keys). `count` with index removal renumbers, breaking all addresses.

▣ Workspaces — same code, isolated state

```
$ terraform workspace list
* default

$ terraform workspace new dev
Created and switched to workspace "dev"!

$ terraform workspace new prod
$ terraform workspace select dev    # switch back
$ terraform workspace show          # current: dev
```

One config, isolated state per workspace. The `default` workspace cannot be deleted. Use workspaces for non-prod isolation; for prod-grade isolation, prefer separate buckets/accounts per env.

▣ count vs for_each — the comparison

count — N identical copies

```
resource "aws_instance" "web" {
  count = 3
  ami   = "ami-123"
  tags  = { Name = "web-${count.index}" }
}
# addresses: web[0], web[1], web[2]
```

Trap: remove `count = 3` → set to 2 → ALL 3 destroy, 2 fresh create. **Don't use for resources with identity.**

for_each — N named copies

```
resource "aws_instance" "web" {
  for_each = toset(["ahmed", "sara", "omar"])
  ami      = "ami-123"
  tags     = { Name = "web-${each.key}" }
}
# addresses: web["ahmed"], web["sara"], web["omar"]
```

Win: remove "ahmed" from the set → only web["ahmed"] destroys. The rest keep their stable keys.

▣ Conditional attributes with for_each

```
variable "create_public_ip" {
  type    = bool
  default = true
}

resource "aws_instance" "web" {
  for_each          = toset(["a", "b", "c"])
  ami               = "ami-123"
  instance_type     = each.key == "a" ? "t3.small" : "t3.micro"
  associate_public_ip_address = var.create_public_ip
}
```

Use `each.key` and `each.value` inside the block. `for_each` accepts a set or a map — set for strings, map for key/value pairs.

⏭️ **Next:** Variables in, outputs out — Outputs.

Outputs — Share, Export, CI/CD

6 MIN · T21

Variables are inputs. **Outputs** are exports — values that surface after `apply` for humans, other modules, and pipelines to consume. Four uses, all in the same `outputs` block.

> 🔑 **THE 4 USES OF OUTPUTS**
> ① Display after apply (read with `terraform output`). ② Share between modules (child exposes, parent consumes). ③ Remote state access via `terraform_remote_state`. ④ CI/CD parsing (raw JSON, scripted pipelines).

Outputs — the full anatomy

```
# outputs.tf
output "vpc_id" {
  value       = aws_vpc.main.id
  description = "the VPC ID — pass to other modules"
  sensitive   = false   # true hides from CLI output
}

output "db_password" {
  value     = aws_db_instance.main.password
  sensitive = true    # won't print in `terraform output`
}

output "all_subnet_ids" {
  value = aws_subnet.public[*].id   # list comprehension
}

# After apply
$ terraform apply
vpc_id = "vpc-0ab123456"
all_subnet_ids = [
  "subnet-0aa...",
  "subnet-0bb...",
  "subnet-0cc...",
]

$ terraform output -raw vpc_id        # no quotes, perfect for scripts
vpc-0ab123456
$ terraform output -json vpc_id     # JSON for CI/CD
{"vpc_id":"vpc-0ab123456"}
```

Sharing outputs across modules

```
# modules/vpc/outputs.tf
output "vpc_id" { value = aws_vpc.main.id }
output "public_subnet_ids" { value = aws_subnet.public[*].id }

# root main.tf — consumer
module "vpc" {
  source = "./modules/vpc"
}

resource "aws_instance" "web" {
  ami       = "ami-123"
  subnet_id = module.vpc.public_subnet_ids[0]   # read module output
}
```

Modules expose values the same way the root does. Reference any output as `module.NAME.OUTPUT`. For sharing state between *separate* Terraform projects (different folders, different teams), use the `terraform_remote_state` data source.

⏭ **Next:** Day 2 closes. Level 2 opens — Providers catalog.

🌍 Providers — IaaS / PaaS / SaaS

6 MIN · T9 (deep)

Providers are **translators**: your HCL speaks `resource "aws_instance" "web" {…}`, the provider translates that into the right HTTP call to AWS, GCP, Cloudflare, or GitHub. 3000+ providers exist; you can write your own for in-house systems.

> 🔑 **THREE LAYERS OF PROVIDERS**
> **IaaS** (raw infra): AWS, GCP, Azure, Alibaba, DigitalOcean. **PaaS** (managed platforms): Heroku, Render, Vercel, Supabase. **SaaS** (third-party APIs): Cloudflare, GitHub, Datadog, PagerDuty, Stripe.

The provider ecosystem at a glance

Layer	Examples	What you manage
IaaS	AWS, GCP, Azure, Alibaba, OCI, DigitalOcean, Linode	VMs, networks, storage, databases (raw primitives)
PaaS	Heroku, Render, Vercel, Fly.io, Supabase	Apps, services, managed databases (higher-level)
SaaS	Cloudflare, GitHub, GitLab, Datadog, PagerDuty, Stripe	DNS, repos, monitoring, alerts, billing
In-house	your own custom provider (Go SDK)	your internal platform's API

One apply, three layers

```
resource "aws_s3_bucket" "logs" { bucket = "study-logs" }      # IaaS
resource "github_repository" "infra" {                           # SaaS
  name = "infra"
  visibility = "private"
}
resource "cloudflare_record" "root" {                              # SaaS
  zone_id = var.cf_zone
  name    = "@"
  value   = aws_s3_bucket.logs.website_endpoint
  type    = "CNAME"
}

$ terraform apply
# one apply, three different APIs called
```

You can call multiple providers in one `apply`. The dependency graph spans all of them. An AWS bucket URL feeds directly into a Cloudflare DNS record. Second region? `provider "aws" { alias = "eu" }`, then `provider = aws.eu` on the resource.

Version constraints — pin the right way

Operator	Means	Example	Verdict
`~> 5.0`	allow 5.x, block 6.0	5.1 → 5.9 ok, 6.0 no	✅ default
`>= 1.6, < 2.0`	range	safe minimum + cap	✅ for CLI
`= 1.9.8`	exact	reproducible	ok for prod freeze
`>= 5.0` alone	no cap	6.0 surprise breaks	❌ avoid

⏭ **Next:** The escape hatch — Provisioners.

🛠 Provisioners — Doctrine: Last Resort

Provisioners run scripts after resource creation or before destruction. They're the escape hatch when the provider doesn't have a feature — and the doctrine is **use them last**. user_data, Packer, Ansible cover 95% better.

⚠️ **THE CRITICAL CAVEAT**

Provisioners run **ONLY on the resource's first creation**. Re-apply with an unchanged resource does NOT re-run them. They re-run only after destroy + recreate. **State never knows if the script succeeded** — only that Terraform "tried". Failed provisioner + existing resource = green plan over a half-built box; recover with `taint` + re-apply. `on_failure = continue` is for best-effort logging only.

▣ local-exec vs remote-exec

local-exec (runs on YOUR machine)

```
resource "aws_instance" "web" {
  ami           = "ami-123"
  instance_type = "t3.micro"

  provisioner "local-exec" {
    command = "echo ${self.private_ip} >>
deployed.txt"
    # runs on the machine running Terraform
  }
}
```

Use cases: register the new instance in your inventory, post to Slack, write to a file. **Does not affect the resource itself.**

remote-exec (runs ON the resource)

```
resource "aws_instance" "web" {
  ami           = "ami-123"
  instance_type = "t3.micro"

  connection {
    type        = "ssh"
    user        = "ubuntu"
    private_key = file("~/.ssh/lab.pem")
    host        = self.public_ip
  }

  provisioner "remote-exec" {
    inline = [
      "sudo apt-get update -y",
      "sudo apt-get install -y nginx"
    ]
    # runs on the new VM via SSH
  }
}
```

Use cases: bootstrap the OS. **Breaks idempotence** if the inline script isn't safe to re-run.

▣ When to use provisioners (the doctrine)

Need	Use this, not a provisioner
Install packages on first boot	`user_data` / `cloud-init` (declarative, retryable, in state)
Bake a golden image	Packer (builds AMI outside Terraform entirely)
Configure the running OS	Ansible (idempotent modules, no SSH key in Terraform)
External side effect (register, post to Slack)	`local-exec` is fine
Something nothing else can do	`remote-exec` as last resort

😅 **THE CLASSIC MISTAKE**

You provision an EC2, then `remote-exec` runs `apt install nginx` on first boot. Plan looks great. Then next week you change the instance type, apply, and... nginx is **still installed** (provisioner doesn't re-run). It works because you got lucky. **The cure:** bake nginx into a Packer AMI and let Terraform deploy images, not scripts.

⏭ **Next:** The Lego brick — Modules.

Modules — Package, Version, Reuse

6 MIN · T23

A module is a **container for multiple resources used together**. Every Terraform configuration is itself a module (the root module). Modules can call other modules — that's reuse, packaging, consistent patterns across teams.

> 🔑 **EVERY CONFIG IS A MODULE**
> You're always in a module. The **root module** = where you run `terraform apply`. **Child modules** = folders you call via `module "x" { source = "..." }`. **Published modules** = on the registry, versioned, immutable.

Module structure

```
modules/ec2/
  main.tf       # resources; no hardcoded values; uses variables
  variables.tf  # all inputs + validations
  outputs.tf    # all exports; never expose secrets unmarked
  versions.tf   # required_version, required_providers
```

Call + read outputs

```
# modules/ec2/main.tf
resource "aws_instance" "this" {
  count         = var.count
  ami           = var.ami
  instance_type = var.instance_type
  tags          = merge(var.tags, { Module = "ec2", Index = count.index })
}

output "instance_ids" { value = aws_instance.this[*].id }

# root: call + read outputs
module "ec2_cluster" {
  source        = "./modules/ec2"
  count         = 3
  ami           = data.aws_ami.ubuntu.id
  instance_type = var.env == "prod" ? "t3.small" : "t3.micro"
}

resource "aws_lb_target_group_attachment" "example" {
  target_group_arn = aws_lb_target_group.app.arn
  target_id        = module.ec2_cluster.instance_ids[0]
}
```

Source	Example	Versioning
Registry	`"terraform-aws-modules/vpc/aws"`	✅ enforced
Git	`"github.com/org/repo//modules/ec2?ref=v1.2.0"`	manual pin
Local	`"./modules/ec2"`	n/a

> ⚠️ **PROVIDER INHERITANCE**
> Reusable modules = **no provider block** (accepts the caller's). `module "vpc" { providers = { aws = aws.eu } ... }` passes a specific provider alias.

⏭ **Next:** The CAPSTONE — Lab 1.

⭐ Lab 1 — Full Build Chain

12 MIN · T24

Stitches every chapter: **Provider** → **VPC** → **Subnets** → **Instances** → **SG** → **Outputs** → **Provisioner** → **Destroy**. Tab 2 builds, Tab 1 observes.

▣ The full main.tf (8 resources, one file)

```
terraform {
  required_providers { aws = { source = "hashicorp/aws", version = "~> 5.0" } }
}

provider "aws" {
  region = "us-east-1"
  access_key = "test"  secret_key = "test"     # Floci: dummy creds; AWS: delete these
  skip_credentials_validation = true
  skip_metadata_api_check     = true
  skip_requesting_account_id  = true
  endpoints { ec2 = "http://localhost:4566" }    # Floci only; delete for AWS
}

resource "aws_vpc" "main" { cidr_block = "10.0.0.0/16" tags = { Name = "capstone" } }

resource "aws_subnet" "a" { vpc_id = aws_vpc.main.id  cidr_block = "10.0.1.0/24"  availability_zone = "us-east-1a" }
resource "aws_subnet" "b" { vpc_id = aws_vpc.main.id  cidr_block = "10.0.2.0/24"  availability_zone = "us-east-1b" }

resource "aws_security_group" "web" {
  name = "web-sg"
  vpc_id = aws_vpc.main.id
  ingress { from_port = 443  to_port = 443  protocol = "tcp"  cidr_blocks = ["0.0.0.0/0"] }
  egress  { from_port = 0    to_port = 0    protocol = "-1"   cidr_blocks = ["0.0.0.0/0"] }
}

resource "aws_instance" "web" {
  for_each               = toset(["a1","a2","b1","b2"])
  ami                    = "ami-0c55b159cbfafe1f0"
  instance_type          = "t3.micro"
  subnet_id              = startsWith(each.key, "a") ? aws_subnet.a.id : aws_subnet.b.id
  vpc_security_group_ids = [aws_security_group.web.id]
  tags                   = { Name = "web-${each.key}" }
}

output "ips" {
  value = { for k, inst in aws_instance.web : k => inst.private_ip }
}

resource "null_resource" "log_ips" {
  triggers = { ids = jsonencode(aws_instance.web) }
  provisioner "local-exec" {
    command = "echo 'instances ready:' >> deployed.txt"
  }
}
```

▣ Two-terminal execution

👀 TAB 1 · OBSERVE

```
floci start
eval "$(floci env)"
watch aws ec2 describe-instances   # watch instances appear
watch aws ec2 describe-vpcs        # watch VPCs
```

⚙ TAB 2 · CONTROL

```
cd ~/lab-capstone
terraform init
terraform plan -out=tfplan
terraform apply tfplan
terraform output -json ips   # → {"a1":"10.0.1.11",...}
terraform destroy -auto-approve
```

🎓 WHAT THIS PROVED

You built the entire chain: provider auth → VPC → 2 subnets → 4 instances via for_each → security group → outputs JSON → local-exec provisioner → full teardown. **The same code runs against real AWS** by deleting the Floci `endpoints` + `skip_*` lines and changing auth to SSO/profile. That's the whole IaC story in 8 resources.

⏭ **Next:** The review — Cheat · Quiz · Glossary · Resources.

FINALE

📋 Cheat Sheet · Quiz · Glossary · Resources

FINALE

The story in one page: commands, quiz, glossary, resources. Five minutes of review before the interview.

▣ Cheat sheet — the 14 commands

Command	What it does
`terraform init`	install providers + wire backend
`terraform fmt`	canonical formatting
`terraform validate`	syntax check (offline)
`terraform plan -out=tfplan`	preview diff, save to file
`terraform apply tfplan`	apply EXACTLY the reviewed plan
`terraform destroy`	tear it all down
`terraform output -raw <name>`	read an output value
`terraform state list / show / mv / rm`	operate on state without recreating
`terraform import <addr> <id>`	adopt existing infra
`terraform force-unlock <ID>`	release a stuck lock (DANGEROUS)
`terraform workspace <list|new|select>`	state isolation per env
`terraform graph | dot -Tsvg`	visualize the dependency graph

▣ Quiz — 8 questions

1. What does `terraform init` do? (jobs 1 & 2)
2. Idempotence in one sentence.
3. The 3 states. What does plan compare?
4. What do + / ~ / -/+ / - mean?
5. Why no keys in `.tf`? (3 reasons)
6. Resource vs data source.
7. Variable precedence: lowest to highest?
8. count vs for_each — which, and why?

🔑 QUICK ANSWERS (OPEN IN YOUR NOTES)

A1. Install providers + wire backend. **A2.** apply twice = apply once. **A3.** Desired vs Known vs Real. **A4.** +/~/−/+create or update or destroy or recreate. **A5.** leaks to state+git+rotation cost. **A6.** managed lifecycle vs read-only lookup. **A7.** defaults < tfvars < -var < TF_VAR_*. **A8.** for_each (stable keys on remove).

Glossary + Resources

Glossary — 20 terms

apply — executes the planned changes.

backend — where `terraform.tfstate` lives.

core — Terraform's brain: parses, graphs, plans, calls plugins.

data source — read-only lookup, no lifecycle.

desired state — what your `.tf` declares.

drift — reality diverged from code.

HCL — HashiCorp Configuration Language.

idempotence — apply twice = apply once.

IGW — Internet Gateway.

known state — what `terraform.tfstate` records.

lock table — DynamoDB row per backend key. Mutex for state writes.

module — reusable, parameterized folder.

NAT Gateway — managed, ~$32/mo+data, AZ-bound.

OIDC — short-lived token CI assumes into IAM roles.

plan — the diff: +/~/−.

provider — plugin that translates HCL to API calls.

provisioner — escape hatch to run scripts. Last resort only.

real state — what the cloud looks like (after refresh).

SSO — temporary credentials, no long-lived keys.

workspace — state isolation per env in one config.

Resources — keep these handy

Course deck (this book is built on it): Eng. Mohamed Magdy — Terraform Level 1 + 2

Official: registry.terraform.io · developer.hashicorp.com/terraform · github.com/hashicorp/terraform
Learn: learn-terraform (free labs) · discuss.hashicorp.com (forum)
Local emulator: floci.io (Floci — free MIT, port 4566)
Exam: HashiCorp Certified — Terraform Associate (covers most of this book)